Eötvös Loránd Tudományegyetem

Informatikai Kar

Média- és Oktatásinformatikai Tanszék

Receptkezelő webalkalmazás

Témavezető:
Dr. Menyhárt László Gábor
adjunktus

Szerző:
Máté Zsolt Sándor
programtervező informatikus BSc

Budapest, 2026

Témabejelentő

Szakdolgozatom témája egy olyan webalkalmazás implementálása, mely lehetővé teszi értékes receptek könnyű és gyors digitalizálását. A receptek megörökítése egy egyszerű és letisztult webes felületen zajlana: az összefoglaló ismertetőt, a hozzávalók listáját és az elkészítés leírását egy űrlap segítségével lehetne rögzíteni. A hozzávalók megadásával automatikusan generálódna egy allergén lista is, mely lehetővé tenné a receptek szűrését a különféle ételintoleranciával rendelkező felhasználók számára. Az allergének szabványos módon, az 1169/2011/EU rendelet alapján kerülnének csoportosításra. Az alkalmazás emellett lehetőséget biztosítana a felhasználók regisztrációjára is, mely által számos további funkció is elérhetővé válna, mint például a felhasználói profil személyre szabása, a saját receptjeik szerkesztése, a böngészésre való lehetőség a recepttárban és a többi felhasználó által feltöltött receptek értékelése. Külön felületet biztosít a profil szerkesztésére, a receptek szerkesztésére és a receptek böngészésére. A szoftver lehetőséget ad az adatok módosítására, amelyek egy relációs adatbázisban kerülnek tárolásra.

Tartalomjegyzék

1. Bevezetés **3**

2. Felhasználói dokumentáció **5**

2.1. Felhasznált módszerek röviden 5

2.2. Rendszerkövetelmények 5

2.3. Telepítés 6

2.3.1. Telepítési útmutató 6

2.4. Funkciók 7

2.4.1. Receptkezelés 7

2.4.2. Közösségi funkciók 7

2.4.3. Keresés 7

2.4.4. Profilkezelés 7

2.5. Az alkalmazás használatának felépítése 8

2.5.1. Kezdőoldal és receptlista 8

2.5.2. Recept részletező oldal 8

2.5.3. Recept létrehozása és szerkesztése 9

2.5.4. Keresés és szűrés 9

2.5.5. Bejelentkezés és regisztráció 10

2.5.6. Profiloldal 10

2.5.7. Adminisztrációs funkciók 11

3. Fejlesztői dokumentáció **12**

3.1. Tervezés 12

3.1.1. Feladat részletes leírása 12

3.1.2. Funkcionális követelmények 12

3.1.3. Nem funkcionális követelmények 13

3.1.4. Felhasználói esetek 13

3.1.5. Felhasználói felület terve 14

3.1.6. Architekturális áttekintés 17
3.1.7. Backend architekturális döntések 19
3.1.8. Frontend architekturális döntések 19
3.1.9. REST API tervezés 19
3.1.10. Adatbázis 23
3.2. Megvalósítás 24
3.2.1. Fejlesztési környezet és eszközök 24
3.2.2. Backend 24
3.2.3. Frontend 25
3.2.4. API szerződés és generálás 25
3.2.5. Fejlesztés közben felmerült problémák 25
3.3. DevOps 25
3.3.1. Docker és konténeres futtatás 25
3.3.2. CI/CD 25
3.4. Tesztelés 26
3.4.1. Unit tesztek 26
3.4.2. Integrációs tesztek 26
3.4.3. Manuális tesztelés 26

4. Összegzés **27**
4.1. További fejlesztési lehetőségek 27

A. Konfigurációs környezeti változók **29**

B. Felhasználói történetek **31**

Irodalomjegyzék **36**

1. fejezet

Bevezetés

A szakdolgozatom célja egy könnyen kezelhető webes alkalmazás létrehozása. Az alkalmazás lehetővé teszi a felhasználók számára, hogy saját receptjeiket feltöltsék, megosszák másokkal, valamint értékeljék és kommentálják a meglévő recepteket. Emellett az alkalmazás támogatja a keresést és a szűrést, különösen nagy hangsúlyt fektetve az allergén információk megjelenítésére, hogy az ételallergiával élő felhasználók számára is könnyedén használható legyen a platform.

Családunk több generáció házi, sokszor kézzel írt receptjeit használja. Ezek a receptek gyakran papíron vagy különböző digitális formátumokban vannak tárolva, de akad olyan eset is, amikor csak szájhagyomány útján terjednek, így követésük és megosztásuk nehézkes lehet. Ezek utólagos módosítása szintén nehezen követhető módon történik. A manapság elérhető népszerű platformok gyakran nem nyújtanak megfelelő támogatást a receptek kezelésére, megosztására és értékelésére, különösen akkor, ha figyelembe vesszük az allergén információk fontosságát. Egy jól megtervezett alkalmazás nemcsak megkönnyítené a receptek megosztását és kezelését, hanem allergén információk megjelenítésével biztonságosabbá is tenné a főzést azok számára, akik ételallergiával élnek.

A dolgozat célja egy olyan könnyen kezelhető webes alkalmazás létrehozása, amely segítségével a felhasználók egyszerűen feltölthetik, megoszthatják és szerkeszthetik saját receptjeiket, kiegészítve közösségi funkciókkal mint például: értékelés, kommentelés. Az alkalmazásnak támogatnia kell a receptek keresését és szűrését, különösen az allergén információk alapján. A szűrési lehetőségek kialakításakor figyelembe kell venni a vendéglátási ipar bevett gyakorlatait, valamint az allergének jelölésére vonatkozó uniós szabályozást, különösen az 1169/2011/EU rendeletet[1].

Ezek kívül egy olyan modern megoldásokat alkalmazó rendszer létrehozása a célom, amely könnyen karbantartható és bővíthető, valamint megfelel a jelenlegi webes fejlesztési szabályoknak és bevett gyakorlatoknak. Emellett fontosnak tartom azt, hogy a rendszer könnyedén elindítható legyen helyi környezetben, és minimalizálja azon külső szoftvereket, amelyek szükségesek a rendszer futtatásához.

2. fejezet

Felhasználói dokumentáció

2.1. Felhasznált módszerek röviden

Az alkalmazás egy webalapú receptkezelő rendszer, amely kliens-szerver architektúrán épül fel. A frontend egy Vue.js[1] alapú egyoldalas alkalmazás, amely a böngészőben fut. A backend egy ASP.NET Core[2] Web API, amely a kéréseket kezeli és az üzleti logikát valósítja meg. Az adatok tárolása PostgreSQL[3] adatbázisban történik. A frontend és a backend közötti kommunikációt egy Nginx[4] fordított proxy koordinálja. Az egész rendszer konténerizált környezetben fut.

2.2. Rendszerkövetelmények

A szerver futtatásához szükséges egy olyan környezet, amin futtatható a Docker[5]. Emiatt az alábbi követelmények szükségesek:

- 64 bit-es Windows 11 vagy Linux operációs rendszer ami támogatja a Docker futtatását
- Legalább 4 GB RAM, ajánlott 8 GB
- Legalább 10 GB szabad tárhely
- Hálózati kapcsolat

[1] `https://vuejs.org/`. Utolsó elérés: 2026.04.10.
[2] `https://dotnet.microsoft.com/en-us/apps/aspnet`. Utolsó elérés: 2026.04.10.
[3] `https://www.postgresql.org/`. Utolsó elérés: 2026.04.10.
[4] `https://nginx.org/`. Utolsó elérés: 2026.04.10.
[5] `https://www.docker.com/`. Utolsó elérés: 2026.04.10.

- Docker Desktop v4.40 vagy frissebb verzió
- Docker Compose v2.35 vagy frissebb verzió

Kliens oldalon egy modern webböngészőre van szükségünk, ami támogatja a HTML5-öt és a JavaScriptet:

- Mozilla Firefox legfrissebb verziója
- Google Chrome legfrissebb verziója
- Microsoft Edge legfrissebb verziója
- Apple Safari legfrissebb verziója

2.3. Telepítés

Az alkalmazást a korábban említett Docker[6], illetve Docker Compose[7] segítségével lehet telepíteni. Ennek köszönhetően egyetlen parancs kiadásával a teljes rendszer elindítható különösebb beállítások nélkül.

2.3.1. Telepítési útmutató

Miután a Docker és a Docker Compose telepítésre került, a projekt mappájába kell navigálni. Az itt található `docker-compose.yml` fájl segítségével indítható el a rendszer. Ez a fájl négy konténerből áll: egy PostgreSQL adatbázisból, a backend REST API-ból, a frontend szerverből és egy Nginx fordított proxyból.

A rendszer konfigurációja egy környezeti változókat tartalmazó fájlon keresztül történik, amelyben a működéshez szükséges beállítások adhatóak meg. Az elérhető konfigurációs opciókat a A.1. és A.2. táblázat tartalmazza.

Elindításához Linux és macOS rendszereken a `start.sh`, Windows rendszereken pedig a `start.ps1` scriptet kell futtatni, amely automatikusan elindítja a szükséges konténereket a konfigurációs fájl alapján. Beállítható az is, hogy a program biztonságos HTTPS kapcsolaton keresztül legyen elérhető, ehhez a konfigurációs fájlban meg kell adni a szükséges tanúsítványokat és kulcsokat.

Az alapbeállításokkal a rendszer a `http://localhost:9393` címen lesz elérhető, amit egy webböngészőben lehet megnyitni.

[6] `https://docs.docker.com/desktop/`. Utolsó elérés: 2026.04.10.
[7] `https://docs.docker.com/compose/install/`. Utolsó elérés: 2026.04.10.

2.4. Funkciók

Az alkalmazásban két fajta felhasználói szerepkör van: adminisztrátor és normál felhasználó. Az oldalon regisztráció nélkül böngészhetőek a receptek. Bejelentkezés után a normál felhasználók hozzáférhetnek a közösségi funkciókhoz, kezelhetik saját receptjeiket, míg az adminisztrátorok további jogosultságokkal rendelkeznek, mint például a felhasználók kezelése.

2.4.1. Receptkezelés

A felület lehetővé teszi a felhasználók számára, hogy könnyedén létrehozzanak, szerkeszthessenek és törölhessenek recepteket. Minden recepthez megadható egy cím, leírás, hozzávalók listája és elkészítési útmutató. Egy képet is feltölthetnek a recepthez.

2.4.2. Közösségi funkciók

A regisztrált felhasználók értékelhetik és kommentelhetik a recepteket. Az értékelés egytől ötig terjedő skálán történik, amelyek csillag formájában jelennek meg. A receptekhez kommenteket is lehet írni, amelyeket a receptek alatt lehet megtekinteni.

2.4.3. Keresés

Az alkalmazás lehetőséget biztosít a receptek közötti keresésre. A keresőmezőbe kulcsszavakat lehet beírni, amelyek alapján a rendszer megjeleníti a releváns recepteket. Emellett a recepttalálatok allergének alapján tovább szűkíthetőek, így a felhasználók könnyedén megtalálhatják a számukra megfelelő recepteket.

2.4.4. Profilkezelés

A felhasználók saját profiljukat is kezelhetik, ahol megtekinthetik és szerkeszthetik profilképüket, adataikat, valamint megváltoztathatják jelszavukat. A profilban láthatóak az általuk létrehozott receptek is, amelyeket könnyen elérhetnek és szerkeszthetnek. Minden profil publikus, amit bármelyik bejelentkezett felhasználó megtekinthet.

2.5. Az alkalmazás használatának felépítése

Az alkalmazás fő eleme az oldal felső részén elhelyezkedő navigációs sáv. Ez a sáv minden oldalon elérhető, és lehetővé teszi a gyors hozzáférést a legfontosabb funkciókhoz. A sávban található egy keresőmező, amely azonnali keresést tesz lehetővé a receptek között, találatokat jelenít meg a kulcsszavak alapján szinte azonnal. Ezen kívül ebben a sávban helyezkednek el gombok a bejelentkezéshez, új recept létrehozásához és összes recept listázására is.

2.5.1. Kezdőoldal és receptlista

A kezdőlapon két kiemelt elem található: egy kiemelt recept szekció, ahol egy előre meghatározott recept jelenik meg, valamint egy recept kirakat, ahol minden nap hat véletlenszerű recept jelenik meg. A kiemelt receptnek láthatjuk a címét, egy rövid leírást és egy képet, valamint a recept értékelését. A recept kirakatban szintén megjelenik a cím, egy rövid leírás és egy kép, de az értékelés nem látható. Mindkét szekcióban kattinthatóak a receptek, így a részletes oldalra navigálhatunk.

Kép helye: kezdőoldal és receptlista nézet

2.1. ábra. A kezdőoldal és a receptlista megjelenése

2.5.2. Recept részletező oldal

Rögtön a recept címe, leírása és képe jelenik meg. A kép felett látható a recept értékelése, amely a korábban említett módon, csillagok formájában jelenik meg. A csillagokra kattintva a recept értékelésére van lehetőség, amely új értékelést ad a recepthez. Egy felhasználó csak egyszer értékelhet egy receptet, de korábbi értékelését módosíthatja.

A leírás alatt kap helyet a hozzávalók listája és az elkészítési útmutató. Könnyen áttekinthető formátumban jelennek meg, ahol a hozzávalók egy felsorolásban, míg az elkészítési útmutató egy számozott listában van megjelenítve. A kép alatt helyezkedik el az allergének listája is.

Ezek alatt található a komment szekció. A kommentek időrendben jelennek meg, a legfrissebb komment van legfelül. Minden kommentben látható a kommentelő neve, a komment szövege és időpontja. A kommentelési blokk legfelső részén jelenik meg a

beviteli mező, ahol a felhasználók új kommentet írhatnak. Ez csak akkor kitölthető, ha a felhasználó be van jelentkezve.

Bejelentkezés után a kép alatt megjelenik kettő gomb, amely lehetővé teszi a recept szerkesztését, illetve törlését, amennyiben a recept létrehozója vagy adminisztrátor vagyunk. A szerkesztés egy új oldalra navigál, ahol módosíthatjuk a recept adatait, hozzávalóit, elkészítési útmutatóját és képét. A törlés gomb megnyomásával egy felugró ablak jelenik meg, hogy biztosan szeretnénk-e törölni a receptet.

Kép helye: recept részletező oldal

2.2. ábra. A recept részletező oldal felépítése

2.5.3. Recept létrehozása és szerkesztése

A receptek létrehozására, illetve szerkesztésére egy ugyanúgy felépített oldalon van lehetőség. Ezen a felületen tudjuk megadni a recept címét, leírását, hozzávalókat, elkészítési útmutatót, illetve tudunk feltölteni képet. A rendszer minden hozzávalóhoz automatikusan megpróbálja kitalálni, hogy milyen allergén információk tartoznak hozzá, de ezeket a felhasználó manuálisan is módosíthatja. Kötelező megadni a címet; leírást; legalább egy darab összetevőt, annak minden elemét; illetve egy elkészítési lépést. Kép feltöltése, illetve további allergének megjelölése opcionális. A hozzávalókhoz megadott allergének alapján készül el a recepthez az allergénlista. Minden beküldendő adat a felületen ellenőrizve van, nem engedi az oldal, hogy hibás adatokkal küldjünk be receptet. A felületen egyértelműen fel van tüntetve, ha a recept készítése közben valamilyen hiba történik.

Kép helye: recept létrehozása és szerkesztése oldal

2.3. ábra. A recept létrehozására és szerkesztésére szolgáló felület

2.5.4. Keresés és szűrés

Az oldal két keresési lehetőséget ad a felhasználónak. Ennek az egyik implementációja a navigációs sávban helyezkedik el. A másik részletesebb keresőt a recepteket listázó oldalon találjuk.

A navigációs sávban egy már ismert kereső helyezkedik el. Itt a recept címei alapján gyorskeresés történik. Amint a felhasználó elkezd beírni legalább 3 karaktert,

az oldal találatokat jelenít meg egy leugró ablakban. Ez úgy lett kialakítva, hogy gyors, szinte azonnali legyen a találatok megjelenítése. A kereső megjeleníti azt is, ha nincs találat.

A másik, összetettebb keresőnél a recept címe mellett allergének alapján lehet szűrni a recepteket. Külön részt kapott, ahol megjelölhetjük azt, hogy mely allergének legyenek benne a keresendő receptekben, illetve melyek maradjanak ki. Ezután az oldalon a csak ezen beállítások alapján leszűrt recepteket böngészhetjük, oldalakba rendezve. Az oldalak közötti váltást a lap alján megjelenő paginációval lehet megvalósítani.

Kép helye: keresés és szűrés felület

2.4. ábra. A keresési és szűrési lehetőségek megjelenése

2.5.5. Bejelentkezés és regisztráció

A navigációs sávban található a bejelentkezés gomb, ami átirányít a bejelentkezés és regisztrációs felületre. Itt már regisztrált felhasználók jelentkezhetnek be email cím és jelszó megadásával. Az oldal felső részén szereplő regisztráció fül megnyomásával lehet a regisztrációs űrlapot megjeleníteni. Meg kell adni a becenevet, email címet, jelszót és jelszónak a megerősítését a regisztrációhoz. Sikeres regisztráció után a felület azonnal be is jelentkeztet minket. Bármilyen hiba esetén az űrlap jelzi, hogy mely mezőkben megadott adattal van probléma. Bejelentkezés után lehet létrehozni receptet, kommentelni receptekhez, értékelni receptet és megtekinteni mások profilját.

Kép helye: bejelentkezési és regisztrációs felület

2.5. ábra. A bejelentkezési és regisztrációs oldal

2.5.6. Profiloldal

A profil nézeten minden adatot megtekinthetünk, ami hozzánk kapcsolódik. Más felhasználóknak csak a profilképet és becenevét látjuk. Saját profilunkon látjuk az email címünket, illetve csillagokkal jelezve a jelszót. A profil alatt megjelenik két gomb, ha a saját profilunkat böngésszük: szerkesztés, törlés. Törlés esetén egy felugró ablak jelenik meg, ahol meg kell erősítenünk a törlési szándékot. Szerkesztés

esetén minden korábban megadott adatot szerkeszthetünk a korábban említett ellenőrzésekkel. Mindenki profilszekciója alatt egy paginációval ellátott recept lista található, ami listázza a felhasználó receptjeit.

Kép helye: profiloldal

2.6. ábra. A profiloldal felépítése

2.5.7. Adminisztrációs funkciók

Az oldalon jelen lévő adminisztrációs funkciók legtöbb esetben külön felületet nem kaptak, hanem néhány korábban említett oldal bővül ki új funkciókkal. A receptek alatt tudnak törölni kommenteket, illetve az adott receptet kiemelheti a főoldalra a „Kiemelt receptté teszem” gombbal. Emellett az adminisztrátor minden receptet és felhasználót törölhet az oldalról. Profil nézetben más felhasználóknak adminisztrátori jogot tudnak adni.

Emelett az adminisztrátorok egy külön hozzávalók oldalon tudják kezelni a hozzávalók alap allergénjeit. Ezen az oldalon egy hozzávaló keresőmező található, ahol a hozzávalók neve alapján lehet keresni. A felület két részből áll: egy kereső, illetve egy szerkesztő rész. A keresőmezőbe beírva a hozzávaló nevét, megjelennek a találatok egy listában. A találatok között lehet váltani, és a kiválasztott hozzávaló adatai megjelennek a szerkesztő részben. Itt módosítható a hozzávaló alap allergénjei, illetve új allergének is hozzáadhatóak, illetve meglévők törölhetőek.

Kép helye: hozzávalók oldal

2.7. ábra. A hozzávalók oldal felépítése

Kép helye: adminisztrációs funkciók megjelenése

2.8. ábra. Az adminisztrációs funkciók megjelenése az alkalmazásban

3. fejezet

Fejlesztői dokumentáció

3.1. Tervezés

3.1.1. Feladat részletes leírása

Az alkalmazás kettő fő részből áll: egy ASP.NET Core alapú backend szolgáltatásból, egy Vue és TypeScript alapú frontend egyoldalas alkalmazásból. Fontos hogy ezen részek önmagukban is megállják a helyüket, de együtt egy teljes funkcionalitást nyújtó rendszert alkotnak. Ennek megvalósásában az stabil API-kapcsolat, a jól definiált adatmodell, valamint a felhasználói élményre fókuszáló frontend kialakítása a fő célok közé tartozik.

3.1.2. Funkcionális követelmények

Funcionális követelmények tekintetében a projekt fő területei a következők:

- Felhasználókezelés: regisztráció, bejelentkezés, profil megjelenítése és módosítása.
- Receptkezelés: új recept létrehozása, szerkesztése, törlése, részletes receptnézet.
- Keresés és szűrés: receptlista böngészése, kulcsszavas keresés, szűrés allergének alapján.
- Közösségi funkciók: receptek értékelése, kommentek rögzítése és törlése.

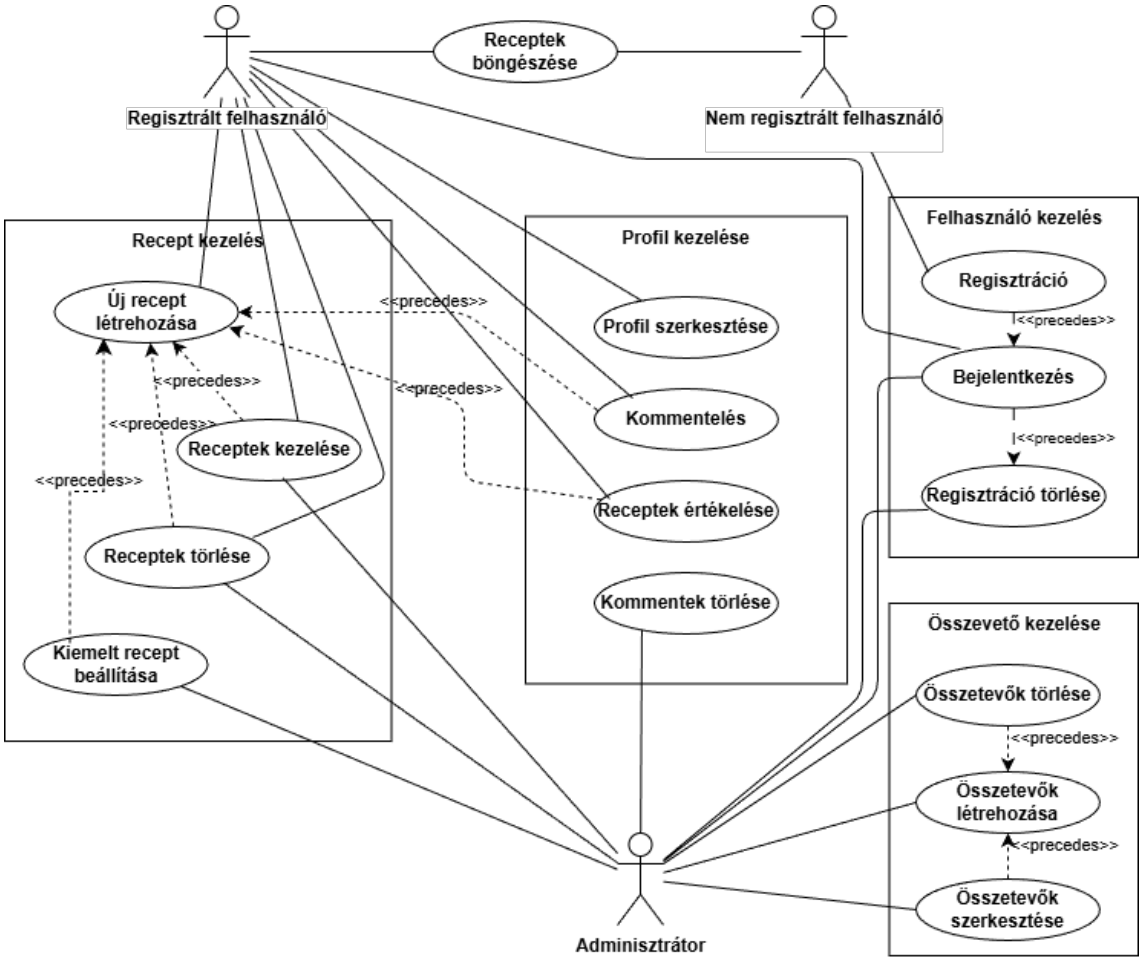
3.1.3. Nem funkcionális követelmények

A rendszerrel szemben a nem funkcionális követelmények a következők:

- átlátható és reszponzív felhasználói felület,
- jól dokumentált, stabilan használható HTTP API,
- olyan adatmodell, amely a recept-, komment- és felhasználói adatok közötti kapcsolatokat konzisztensen kezeli,
- könnyen továbbfejleszthető frontend és backend oldalon,
- fejlesztői munkát segítő konténeres indítási és build környezet [2].
- a rendszer helyi környezetben való könnyű indíthatósága, minimális külső függőséggel.

3.1.4. Felhasználói esetek

Az alábbi ábrán jól látható a rendszer főbb funkciói és azok kapcsolatai. A diagramon a főbb szereplők (felhasználó, adminisztrátor) és azok interakciói jelennek meg a rendszer különböző részeivel. Minden adminisztrátor egyben az alap felhasználó funkcióival is rendelkezik, ez külön nincs feltüntetve a diagramon a könnyebb átláthatóság érdekében.

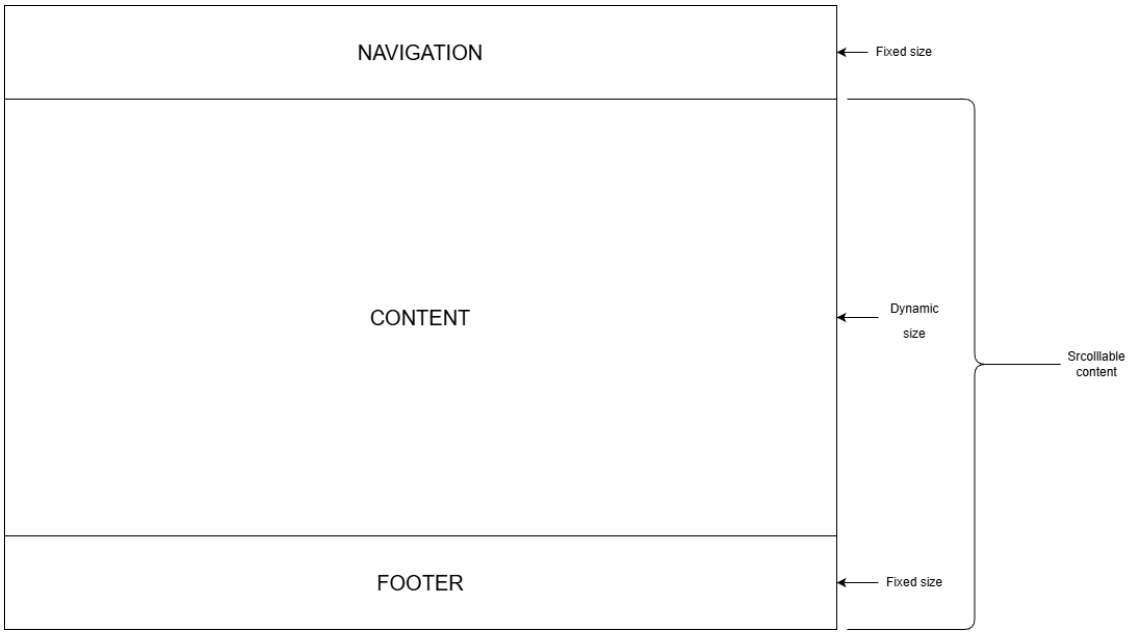


3.1. ábra. Felhasználói esetek diagramja

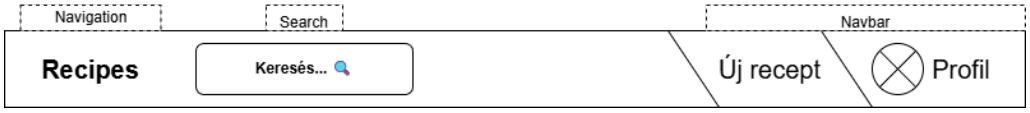
A felhasználói történetek a B függelékben találhatóak meg. Részletesen, táblázatban összefoglalva.

3.1.5. Felhasználói felület terve

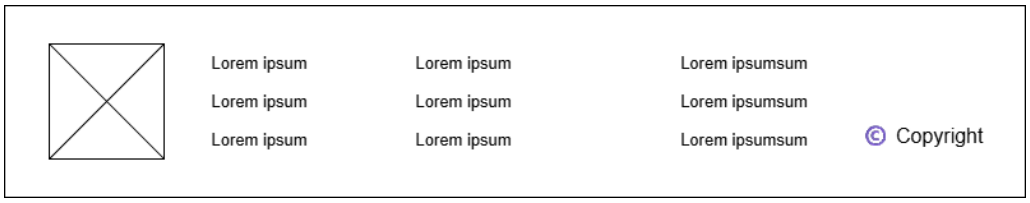
A felhasználói felület vázát DrawIO segítségével készítettem el. Fontos szempont volt, hogy ezek a tervek kellő rugalmasságot biztosítsanak, így a későbbiekben könnyen igazíthatók legyenek a felhasználói igényekhez. A végleges megvalósítás kisebb mértékben eltérhet a kezdeti tervektől, azonban az alapvető struktúra és a főbb komponensek elrendezése megmaradt. Egy oldal felépítése és a főbb komponensek elhelyezése a következőképpen alakul:



3.2. ábra. Oldal elrendezés vázlata

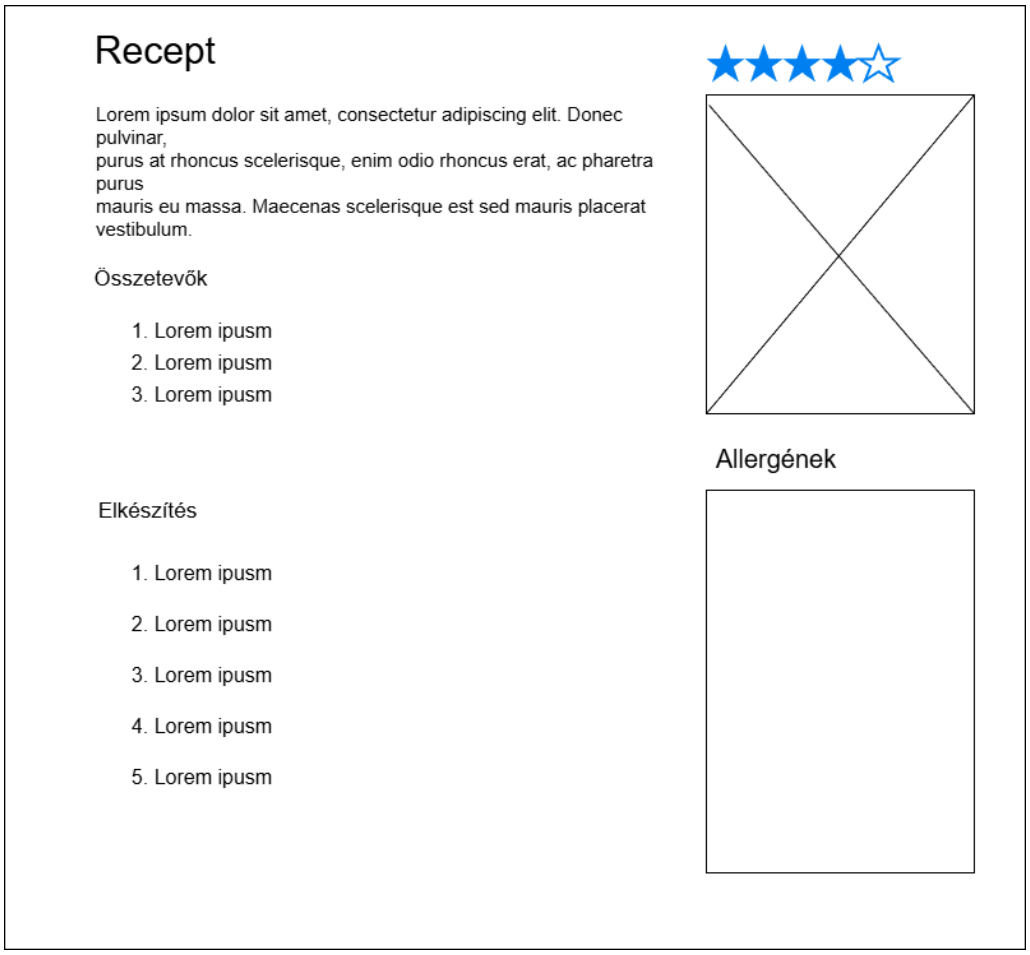


3.3. ábra. Navigációs sáv vázlata



3.4. ábra. Lábléc vázlata

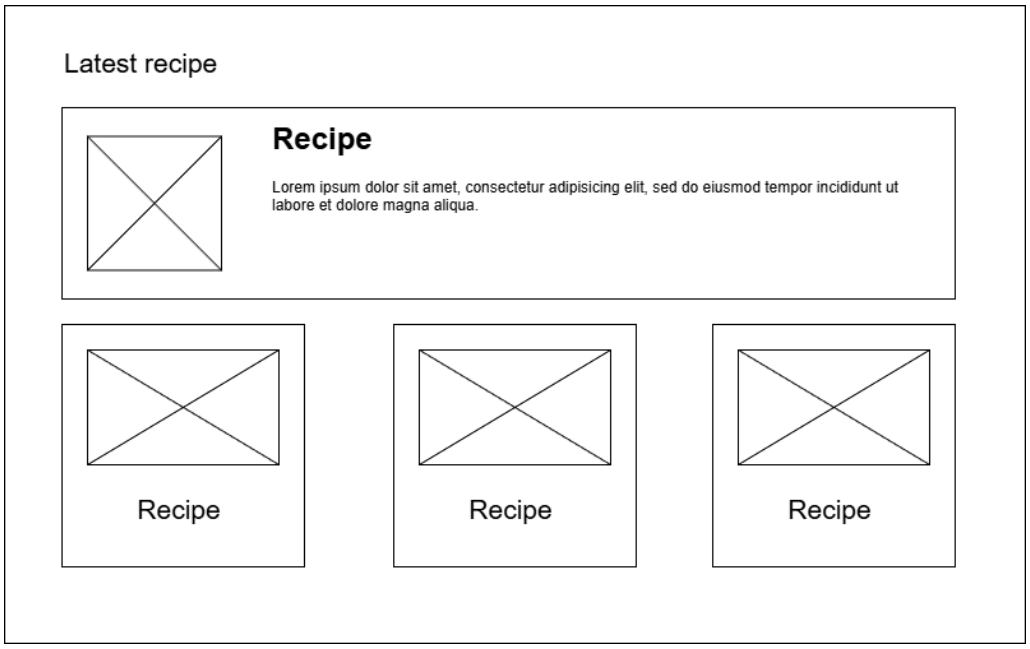
A receptek részletes nézetének vázlatát a 3.5 ábrán látható. Ez az oldal különösen fontos, hiszen itt jelenik meg a recept összes információja és az értékelések is.



3.5. ábra. Recept részletes nézet vázlata

A profil oldal vázlata a 3.6 ábrán, míg a kiemelt receptek oldal vázlata a 3.7 ábrán látható.

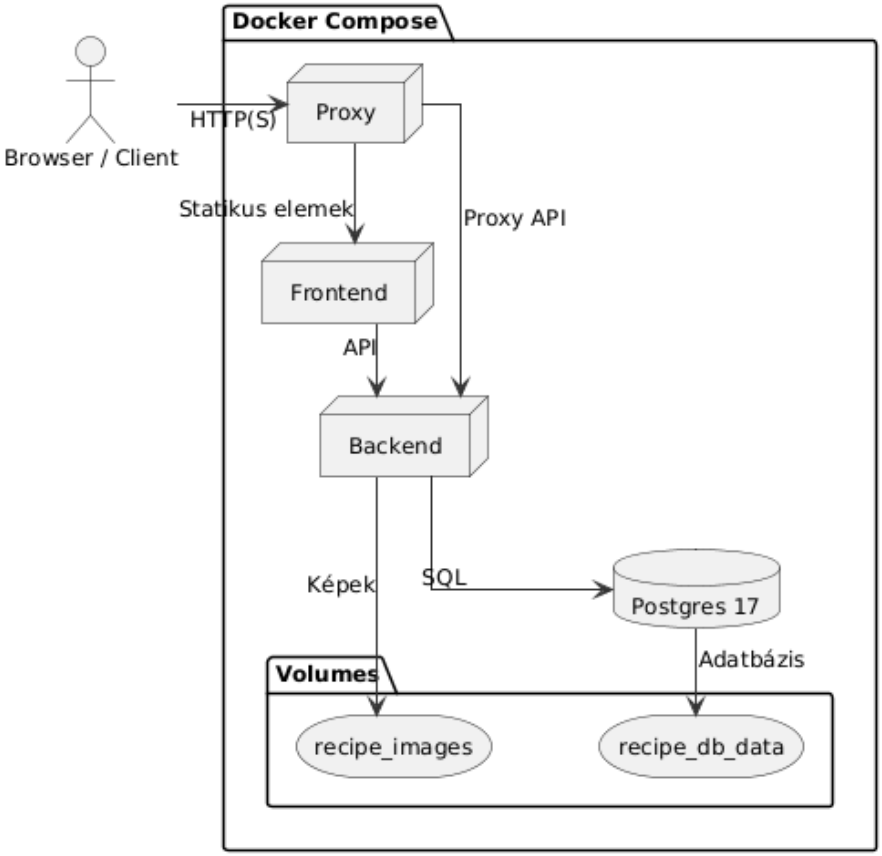
3.6. ábra. Profil oldal vázlata



3.7. ábra. Kiemelt receptek és recept kirakat oldal vázlata

3.1.6. Architekturális áttekintés

A projekt réteges, webalkalmazás-jellegű architektúrára épül. A felhasználó egy böngészőben futó kliensalkalmazással lép kapcsolatba, amely HTTP hívásokon keresztül éri el a backend szolgáltatást. A backend végzi az üzleti logika, az adatelérés, a hitelesítéshez kapcsolódó folyamatok, valamint az API-dokumentáció kiszolgálásának fő részét. Az adatok perzisztálását egy relációs adatbázis biztosítja [3, 4, 5].



3.8. ábra. Rendszer architektúrája

A fő komponensek:

- Frontend: Vue 3 alapú egyoldalas alkalmazás, amely a felhasználói folyamatokat és az oldalak közti navigációt kezeli.
- API kliens: OpenAPI alapján generálható klienscsomag, amely a frontend és a backend közötti szerződést teszi követhetővé.
- Backend: ASP.NET Core Web API, amely kontroller-, szolgáltatás- és repository-szintű felelősségekre tagolódik
- Adatbázis: PostgreSQL adatbázis, amely a receptek, kommentek, felhasználói adatok és egyéb kapcsolódó információk tárolásáért felelős.
- Proxy: Nginx alapú reverse proxy, amely a frontend és a backend szolgáltatásokat egyetlen végpont alá helyezi
- Docker volumes: az adatbázis adatai és a képeket egy-egy megosztott kötetben tároljuk.

3.1.7. Backend architekturális döntések

A backend architektúra réteges felépítésű. A kontroller réteg felelős a HTTP kérések fogadásáért, a válaszok visszaküldéséért, valamint a bemeneti adatok validálásáért. A szolgáltatásréteg tartalmazza az üzleti logikát, és a kontroller által meghívott műveleteket hajtja végre. A repository réteg az adatbázis-műveletekért felel, elválasztva az adatkezelést az üzleti logikától. Az adatok egy entitásmodell segítségével kerülnek reprezentálásra, amelyeket relációs adatbázisban tárolunk. Ez a megközelítés elősegíti a kód tisztaságát, a felelősségek elkülönítését és a rendszer karbantarthatóságát.

3.1.8. Frontend architekturális döntések

A frontend architektúra komponensalapú felépítést követ. A Vue 3 keretrendszer használata lehetővé teszi újrafelhasználható, paraméterezhető komponensek létrehozását. A felület nézetkomponensekre van bontva, amelyek az egyes oldalak megjelenítéséért felelősek. A közös elemek, mint például a navigációs sáv és a lábléc, külön komponensként vannak definiálva. Az állapotkezelést egy központi store (Pinia) biztosítja, amely lehetővé teszi az adatok megosztását a különböző komponensek között. A frontend a backenddel REST API-n keresztül kommunikál. Ez a megközelítés biztosítja a frontend és a backend közötti laza csatolást, valamint megkönnyíti a fejlesztést és a tesztelést.

3.1.9. REST API tervezés

Az alkalmazás tervezésénél fontos szempont volt, hogy a backend és a frontend önmagukban is teljes értékű, jól működő komponensek legyenek. Ennek eléréséhez elengedhetetlen, hogy a két rendszer jól dokumentált és következetes kommunikációt folytasson egymással. Ez lehetővé teszi a frontend és a backend párhuzamos fejlesztését, valamint azt, hogy a komponensek külön-külön is könnyen tesztelhetők legyenek.

Erre a problémára a Representational State Transfer (REST) API tervezési megközelítés nyújt megoldást, amely egy széles körben alkalmazott szabvány a webes szolgáltatások kialakításában. A REST API-k használata biztosítja a frontend és a backend közötti egyszerű, egységes és hatékony kommunikációt.

A megközelítés alkalmazása számos előnnyel jár: támogatja a rendszer skálázhatóságát, megkönnyíti a tesztelést, valamint lehetővé teszi a komponensek egymástól független fejlesztését és karbantartását. A backend szolgáltatás nem függ közvetlenül a frontend implementációjától, így a két réteg lazán csatolt módon működik együtt.

További előny, hogy a rendszer könnyen kiterjeszthető más platformokra is. Amennyiben a jövőben mobilalkalmazás vagy egyéb kliens kerül bevezetésre, ezek ugyanazt az API-t használhatják, így elegendő csupán a megjelenítési réteget az adott platformhoz igazítani.

Előnyök mellett sajnos hátrányokkal is jár. REST API-k tervezése időigényes lehet, különösen összetett üzleti logika esetén. Emellett növeli komplexitást, több eszközt kell használni fejlesztéshez és több programozási nyelvet kell ismerni.

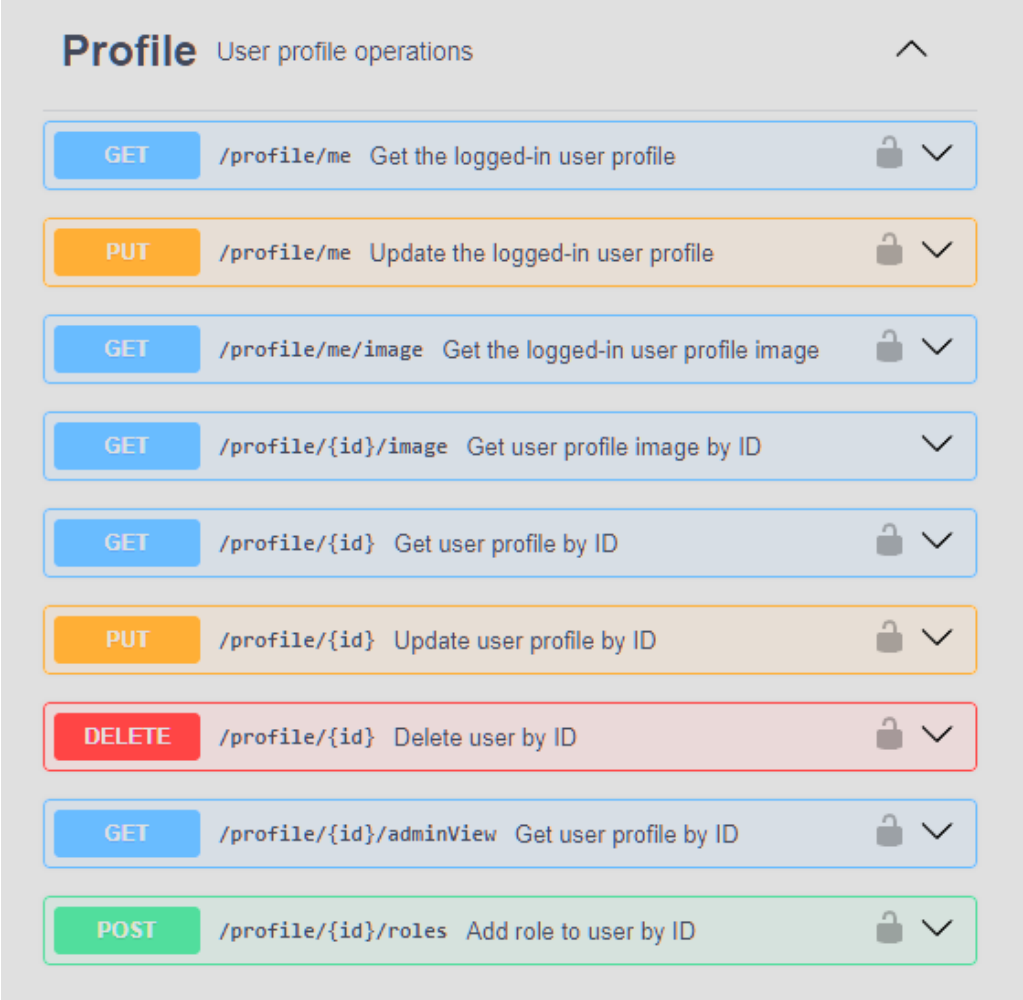
Az api leírására az OpenAPI specifikációval lett megvalósítva. Ez egy széles körben használt szabvány, amely lehetővé teszi a REST API-k részletes dokumentálását. A HTTP végpontok leírása a következőképpen néz ki:

Recipes Recipe operations

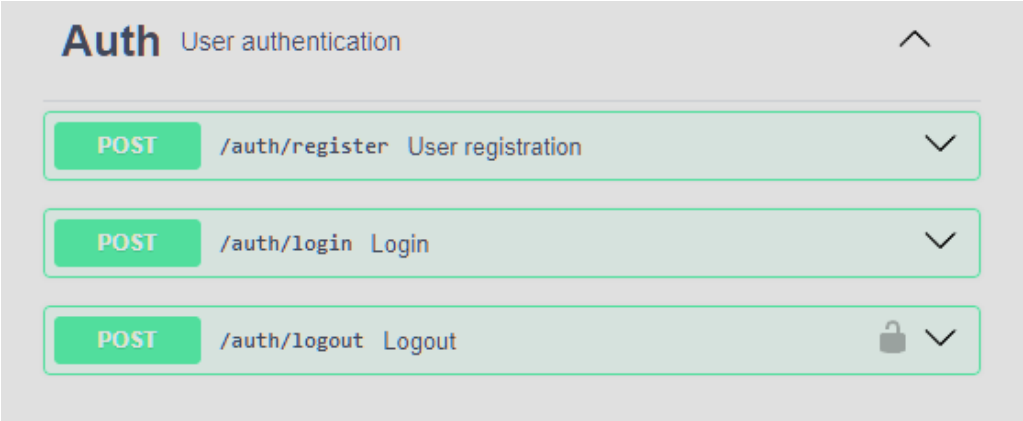
GET	/search	Search recipes by title
GET	/profile/{id}/recipes	List recipes by user ID
GET	/recipes/	List recipes
POST	/recipes/	Create a new recipe
GET	/recipes/{id}/	Get recipe by ID
DELETE	/recipes/{id}/	Delete recipe by ID
PUT	/recipes/{id}/	Update recipe by ID
PUT	/recipes/{id}/rate	Rate recipe by ID (1-5 stars)
PUT	/recipes/{id}/image	Update recipe image by ID
GET	/recipes/{id}/image	Get recipe image by ID
POST	/recipes/{id}/comments	Add comment to recipe by ID
GET	/recipes/{id}/comments	List comments for recipe by ID
DELETE	/recipes/{recipeId}/comments/{commentId}	Delete recipe comment by ID
GET	/recipes/showcase	List featured recipes
GET	/recipes/featured	Get featured recipe
PUT	/recipes/featured	Update featured recipe

3.9. ábra. Receptek HTTP végpontjai

3.10. ábra. Hozzávalók HTTP végpontjai



3.11. ábra. Felhasználói profil HTTP végpontjai



3.12. ábra. Hitelesítési HTTP végpontok

3.1.10. Adatbázis

Az adatbázis tervezése során a fő szempont az volt, hogy minél kevesebb interakció törénjen az adatbázissal natív SQL nyelven írt lekérdezésekkel. Ennek megoldására az Entity Framework (továbbiakban EF) keretrendszert használj a backend. Az EF egy objektum-relációs leképező (ORM) eszköz, amely lehetővé teszi, hogy a fejlesztők C# osztályokkal dolgozzanak, miközben az adatok relációs adatbázisban tárolódnak. Ennek köszönhetően az adatok kapcsolatait és a lekérdeséseket is a C# kódon keresztül lehet kezelni, anélkül, hogy közvetlenül SQL-t kellene írni. Ennek köszönhetően az adatbázis struktúrája típusos, jól karbantartható. A kapcsolatokat a nyelv megszokott kulcsszavaival lehet kezelni.

3.2. Megvalósítás

3.2.1. Fejlesztési környezet és eszközök

Fejlesztői környezetnek több szoftvert és eszközt is használtam. A backend fejlesztéséhez Visual Studiot használtam, frontendhez Visual Studio Code-ot. Ezek a szoftverek könnyedén hozzáférést biztosítanak a szükséges bővítményekhez, mint például az Entity Framework Core eszközei, a Vue támogatás, valamint a Docker integráció.

A fejlesztéshez szükséges a már korábban említett Docker, Docker Compose, ami a konténerizációval a rendszer futtatásához szükséges.

Verziókezéléshez Git-en keresztül történik, a repository pedig a GitHubon található.

A backendben a függőségek NuGet csomagkezelővel vannak kezelve. ASP.NET Core 10-re épül a projekt, és olyan csomagokat használ, mint az Entity Framework Core, valamint a Microsoft Identity platformhoz kapcsolódó csomagok.

Frontend oldalon a Vite és a Vue 3 használata mellett TypeScript is jelen van, ami a típusbiztonságot és a fejlesztési élményt javítja. A csomagok pnpm-en keresztül vannak kezelve, és a package.json fájlban vannak felsorolva. A project egyéb részén is pnpm van használva, például a frontend build és tesztelési parancsoknál.

E2E teszteléshez Cypress-t történik, amely a frontend és backend integrációját teszteli.

A REST API specifikáció alapján OpenAPI Generator CLI generál egy frontend API klienst, és egy backend projectet, hogy a fejlesztés során a backend és frontend szinkronban legyenek. Ehhez szükséges Java 17 vagy annak újabb verziója.

Az adatbázis megtekintéséhez Adminer van bekötve a konténeres környezetbe, amely egy könnyen használható webes adatbázis-kezelő eszköz.

A könnyű fejlesztés érdekében a legtöbb gyakran használt parancs, mint például a build, a tesztelés, vagy a konténeres futtatás, PowerShell szkriptek segítségével van automatizálva.

3.2.2. Backend

A backend ASP.NET Core alapokra épül, és kontrollerek, szolgáltatások, repository-k, valamint adatbázis-kezeléssel kapcsolatos komponensek közműködé-

séből áll. A projektben megjelenik az Entity Framework Core használata, az automatikus migrációfuttatás, a Swagger alapú API-leírás, valamint a hitelesítési és jogosultságkezelési infrastruktúra.

3.2.3. Frontend

A frontend Vue és TypeScript alapokra épülő egyoldalas alkalmazás. A komponensalapú felépítés, a kliensoldali routing, valamint az API klienssel való integráció lehetővé teszi, hogy a felhasználói folyamatok jól elkülöníthető modulokban jelenjenek meg [3]. A felületnek különösen fontos szerepe van a receptlista, a részletes receptnézet, a keresési felület, valamint a profil- és kommentfunkciók megjelenítésében.

3.2.4. API szerződés és generálás

Az OpenAPI specifikáció külön értéket képvisel a projektben, mert egyszerre támogatja a backend dokumentálását, a kliensoldali integrációt és a következetes interfészek kialakítását. A dolgozatban itt lehet bemutatni, hogy a specifikáció hogyan kapcsolódik a generálható klienshez, és ez milyen módon csökkenti a frontend és backend közötti eltérések kockázatát [6].

3.2.5. Fejlesztés közben felmerült problémák

3.3. DevOps

3.3.1. Docker és konténeres futtatás

A repositoryban szereplő Dockerfile-ok és Docker Compose állományok arra utalnak, hogy a projekt fejlesztéséhez és terjesztéséhez konténeres workflow is rendelkezésre áll. Ebben az alfejezetben érdemes bemutatni a fő szolgáltatások együttfutásának módját, valamint a fejlesztői és publikációs konfigurációk közötti eltéréseket [2].

3.3.2. CI/CD

3.4. Tesztelés

A projektben a validáció több szinten értelmezhető. Egyrészt igazolni kell, hogy a rendszer a definiált use case-ek alapján működőképes. Másrészt technikai oldalról is be kell mutatni, hogy a frontend, a backend és az API-kapcsolat stabilan együtt tud működni.

3.4.1. Unit tesztek

3.4.2. Integrációs tesztek

3.4.3. Manuális tesztelés

4. fejezet

Összegzés

Összességében úgy érzem, sikerült minden fontosabb követelményt megvalósítanom. Először dolgoztam ASP.NET Core rendszerrel, így a backend fejlesztése különösen tanulságos volt számomra. Sokat fejlődött a generikus kód írási képességem is. A szigorú alapkövetelményeknek köszönhetően könnyedén tudtam bővíteni a a rendszert funkciókkal.

Lefontosabb célkitűzésem azt volt, hogy a szofter nagyon könnyedén indítható legyen helyi környezetben, minimális külső függőséggel. A szakdolgozat írás közben ismerkedtem meg jobban a GitHub, GitHub Actions és a Docker használatával, így a fejlesztői környezet kialakítása során ezekre a technológiákra támaszkodtam. Ennek köszönhetően a kód felhőben tárolva, bizonságosan verziókezelve van. A GitHub Actions segítségével pedig egy teljesen automatizált build és tesztelési folyamatot hoztam létre.

Szeretnék köszönetet nyílvántítani Kovács Sándor Márknak a folyamatos támogatásért, a hasznos tanácsokért és a szakmai útmutatásért. Sok időmbe telt, mire megírtam ezt a dolgozatot, és a segítsége nélkül nem lett volna lehetséges. Köszönöm a türelmét és a biztatását páromnak, volt csoportársaimnak és barátaimnak, akik végig mellettem álltak, és segítettek megtartani a motivációmat.

4.1. További fejlesztési lehetőségek

A webalkalmazás jelenlegi állapotában is teljes funkcionalitást nyújt, azonban számos lehetőség van a további fejlesztésre. Néhány lehetséges irány:

- E-mail szolgáltatás integrálása a regisztráció és jelszó visszaállítás folyamatába.
- Közösségi média integráció, például Google vagy Facebook bejelentkezés.
- Cimkézés és kategorizálás a receptekhez, hogy könnyebben lehessen keresni és szűrni őket.
- Receptajánló rendszer fejlesztése, amely a felhasználói preferenciák és korábbi aktivitás alapján ajánl recepteket.
- Mobilalkalmazás fejlesztése a webalkalmazás mellett
- Keresési és szűrési funkciók továbbfejlesztése, például összetettebb szűrők

A. függelék

Konfigurációs környezeti változók

Kulcs	Érték	Jelentés
`APP_NAME`	`my-recipe-app`	Az alkalmazás megjelenített neve.
`APP_HOST`	`127.0.0.1`	A webszerver által figyelt hosztcím vagy IP-cím.
`APP_PORT`	`9393`	Az alkalmazás hálózati portja.
`ADMIN_EMAIL`	`recipe@example.com`	Az alapértelmezett adminisztrátori fiók e-mail-címe.
`ADMIN_DEFAULT_PASSWORD`	`Admin123!`	Az adminisztrátori fiók kezdeti jelszava.
`DATABASE_USERNAME`	`admin`	Az adatbázis-kapcsolathoz használt felhasználónév.
`DATABASE_PASSWORD`	`admin`	Az adatbázis-kapcsolathoz tartozó jelszó.

A.1. táblázat. A rendszer működéséhez szükséges kötelező konfigurációs környezeti változók

Kulcs	Érték	Jelentés
PROXY_TLS_ENABLED	true	Meghatározza, hogy a fordított proxy használ-e TLS-t.
PROXY_TLS_CERTIFICATE_FILE	tls.crt	A TLS-tanúsítvány fájlneve vagy elérési útja.
PROXY_TLS_CERTIFICATE_KEY_FILE	tls.key	A TLS-privát kulcs fájlneve vagy elérési útja.
SIZE_OPTION_THUMBNAIL_HEIGHT	50	A miniatűrnézeti képmagasság pixelben.
SIZE_OPTION_THUMBNAIL_WIDTH	50	A miniatűrnézeti képszélesség pixelben.
SIZE_OPTION_MEDIUM_HEIGHT	200	A közepes méretű képmagasság pixelben.
SIZE_OPTION_MEDIUM_WIDTH	200	A közepes méretű képszélesség pixelben.
SIZE_OPTION_LARGE_HEIGHT	500	A nagy méretű képmagasság pixelben.
SIZE_OPTION_LARGE_WIDTH	500	A nagy méretű képszélesség pixelben.
Lapozási beállítások		
PAGINATION_DEFAULT_PAGE_SIZE	12	Az egy oldalon megjelenített receptek alapértelmezett száma.

A.2. táblázat. A rendszerhez tartozó opcionális konfigurációs környezeti változók

B. függelék

Felhasználói történetek

Felhasználó regisztráció

	Leírás	**Kód**
GIVEN	A látogató nincs bejelentkezve és megnyitotta a regisztrációs oldalt.	U01A
WHEN	Kitölti a becenév, e-mail, jelszó és jelszó megerősítése mezőket, majd beküldi az űrlapot.	
THEN	A rendszer létrehozza a fiókot, sikeres visszajelzést ad és automatikusan bejelentkezteti a felhasználót.	
GIVEN	A látogató nincs bejelentkezve és olyan e-mail címmel tölti ki az űrlapot, amely már használatban van.	U01B
WHEN	Beküldi a regisztrációs űrlapot.	
THEN	Hibaüzenet: az e-mail cím már foglalt.	
GIVEN	A látogató nincs bejelentkezve és az űrlapon érvénytelen e-mail címet vagy nem megfelelő jelszót ad meg.	U01C
WHEN	Beküldi az űrlapot.	
THEN	Hibaüzenetek jelennek meg a hibás mezők mellett.	

Új recept hozzáadása

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve és jogosult receptet létrehozni.	U02A
WHEN	Megnyitja az "Új recept" oldalt, megadja a kötelező adatokat (cím, leírás, legalább egy hozzávaló és legalább egy elkészítési lépés), opcionálisan képet tölt fel, majd beküldi az űrlapot.	
THEN	A rendszer ellenőrzi az adatokat, létrehozza a receptet és megjeleníti a recept részletező oldalát.	
GIVEN	A felhasználó be van jelentkezve, de a recept űrlap hiányos vagy érvénytelen adatokat tartalmaz.	U02B
WHEN	Beküldi az űrlapot.	
THEN	A rendszer nem menti el a receptet, hibaüzeneteket jelenít meg és lehetőséget ad a javításra.	

Recept keresése

	Leírás	Kód
GIVEN	A felhasználó eléri a webalkalmazást és megnyitotta a kereső- vagy receptlista felületet.	U03A
WHEN	Keresési kulcsszavakat ad meg vagy allergén alapú szűrőket állít be.	
THEN	A rendszer megjeleníti az illeszkedő recepteket.	
GIVEN	A felhasználó olyan kulcsszót vagy szűrőfeltételt adott meg, amelyhez nem tartozik találat.	U03B
WHEN	A keresés lefut.	
THEN	"Nincs találat" üzenet jelenik meg; a felhasználó módosíthatja a keresést.	

Recept értékelése

	Leírás	Kód
GIVEN	A felhasználó be van jelentkezve, és megnyitotta egy recept részletező oldalát.	U04A
WHEN	Kiválaszt egy 1 és 5 közötti értékelést.	
THEN	A rendszer elmenti vagy frissíti a felhasználó értékelését, és frissíti az átlagértékelést.	
GIVEN	A felhasználó be van jelentkezve, és korábban már értékelte a receptet.	U04B
WHEN	Másik csillagértéket választ a recepthez.	
THEN	A rendszer frissíti a korábbi értékelést, és az új átlagértékelést jeleníti meg.	

Recept részleteinek megtekintése

	Leírás	Kód
GIVEN	A felhasználó kiválaszt egy receptet a listából, a kezdőoldalról vagy a keresési találatok közül.	U05A
WHEN	Megnyitja a recept részletei oldalt.	
THEN	A rendszer betölti és megjeleníti a recept teljes adatait (cím, leírás, hozzávalók, elkészítés, allergén információk, értékelések, hozzászólások).	
GIVEN	A felhasználó olyan receptazonosítóra navigál, amelyhez nem tartozik létező recept.	U05B
WHEN	Megnyitja a recept részletei oldalt.	
THEN	Hibaüzenet és visszairányítás a receptlistához.	

Recept szerkesztése

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve, és a recept tulajdonosa vagy adminisztrátor.	U06A
WHEN	Megnyitja a szerkesztő űrlapot, módosítja az adatokat és beküldi.	
THEN	A rendszer ellenőrzi és frissíti a receptet az adatbázisban, majd megjeleníti a módosított receptet.	
GIVEN	A felhasználó jogosult a szerkesztésre, de érvénytelen vagy hiányos adatokat ad meg.	U06B
WHEN	Beküldi a módosításokat.	
THEN	A rendszer nem menti el a módosításokat, hibaüzeneteket jelenít meg és lehetőséget ad a javításra.	

Recept törlése

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve, és a recept tulajdonosa vagy adminisztrátor.	U07A
WHEN	Kiválasztja a törlés opciót és megerősíti a műveletet.	
THEN	A rendszer törli a receptet és frissíti a listát; megerősítést jelenít meg.	
GIVEN	A felhasználó be van jelentkezve, és jogosult a recept törlésére.	U07B
WHEN	Elindítja a törlést, de a megerősítő ablakban megszakítja a műveletet.	
THEN	A rendszer megszakítja a törlést, a recept megmarad.	

Saját profil megtekintése

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve.	U08A
WHEN	Megnyitja a saját profiloldalát.	
THEN	A rendszer megjeleníti a saját adatait (becenév, e-mail, profilkép, létrehozott receptek listája), valamint a profilhoz tartozó műveleteket.	
GIVEN	A felhasználó be van jelentkezve, és egy másik felhasználó profilját nyitja meg.	U08B
WHEN	Megnyitja a másik felhasználó profiloldalát.	
THEN	A rendszer csak a nyilvános adatokat jeleníti meg (becenév, profilkép, létrehozott receptek listája).	

Felhasználói profil frissítése

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve és megnyitotta a saját profil szerkesztő felületét.	U09A
WHEN	Módosítja adatait és beküldi az űrlapot.	
THEN	A rendszer ellenőrzi és frissíti a profiladatokat, majd megerősítést jelenít meg.	
GIVEN	A felhasználó be van jelentkezve, de érvénytelen bemenetet ad meg (pl. hibás e-mail formátum).	U09B
WHEN	Beküldi a módosításokat.	
THEN	A rendszer nem menti el a módosításokat, hibaüzenetet jelenít meg, a felhasználó javíthatja az adatokat.	

Hozzászólás hozzáadása

	Leírás	**Kód**
GIVEN	A felhasználó be van jelentkezve, és megnyitotta egy recept részletező oldalát, ahol elérhető a hozzászólásmező.	U10A
WHEN	Beír egy hozzászólást és beküldi.	
THEN	A rendszer elmenti a hozzászólást és megjeleníti azt a recept oldalon.	
GIVEN	A felhasználó be van jelentkezve, de üres vagy érvénytelen hozzászólást adott meg.	U10B
WHEN	Megpróbálja beküldeni.	
THEN	A rendszer nem menti el a hozzászólást, hibaüzenet jelenik meg, a felhasználó javíthatja a bejegyzést.	

Hozzászólás törlése

	Leírás	**Kód**
GIVEN	Az adminisztrátor be van jelentkezve, és a hozzászólás létezik.	U11A
WHEN	Kiválasztja a törlés opciót és megerősíti.	
THEN	A rendszer törli a hozzászólást és eltávolítja a megjelenítésből.	
GIVEN	Az adminisztrátor be van jelentkezve, és egy létező hozzászólás törlését kezdeményezte.	U11B
WHEN	A megerősítő ablakban megszakítja a törlést.	
THEN	A hozzászólás megmarad.	

Összetevő keresése

	Leírás	**Kód**
GIVEN	Az adminisztrátor be van jelentkezve, és megnyitotta az összetevőkezelő felületet.	U12A
WHEN	Egy összetevő nevére keres.	
THEN	A rendszer megjeleníti a találatokat, és a kiválasztott összetevő adatait betölti a szerkesztő részbe.	
GIVEN	Az adminisztrátor be van jelentkezve, de üres keresőkifejezést ad meg.	U12B
WHEN	Keresést indít.	
THEN	A rendszer nem indít érvénytelen keresést, és megfelelő visszajelzést jelenít meg.	
GIVEN	Az adminisztrátor be van jelentkezve, és egy meglévő összetevő szerkesztő nézetét látja.	U12C
WHEN	Módosítja az összetevő alap allergén információit és elmenti a változtatásokat.	
THEN	A rendszer érvényesíti és elmenti a módosításokat, majd frissíti a megjelenített adatokat.	

Irodalomjegyzék

[1] *Regulation (EU) No 1169/2011 of the European Parliament and of the Council on the provision of food information to consumers.* Official Journal of the European Union. 2011. URL: https://eur-lex.europa.eu/eli/reg/2011/1169/oj (elérés dátuma 2026. 04. 09.).

[2] *Docker documentation.* Docker, Inc. 2026. URL: https://docs.docker.com/ (elérés dátuma 2026. 04. 09.).

[3] *Vue.js documentation.* Vue.js. 2026. URL: https://vuejs.org/guide/ (elérés dátuma 2026. 04. 09.).

[4] *ASP.NET Core documentation.* Microsoft. 2026. URL: https://learn.microsoft.com/aspnet/core/ (elérés dátuma 2026. 04. 09.).

[5] *PostgreSQL documentation.* The PostgreSQL Global Development Group. 2026. URL: https://www.postgresql.org/docs/ (elérés dátuma 2026. 04. 09.).

[6] *OpenAPI Initiative.* Linux Foundation. 2026. URL: https://www.openapis.org/ (elérés dátuma 2026. 04. 09.).